



KAPITEL

5

Funktionale Verarbeitung von Sammlungen (fortgeschrittene Konzepte)

Lernziele

Zentrale Konzepte in diesem Kapitel: Funktionale Programmierung, Lambda-Ausdrücke, Streams, Pipelines

Java-Konstrukte in diesem Kapitel: Stream, Lambda-Ausdruck, `filter`, `map`, `reduce`, `->`

In diesem Kapitel führen wir die ersten der fortgeschrittenen Konzepte ein. Abschnitte dieser Art (es wird später noch einige mehr geben) können beim ersten Lesen getrost übersprungen werden. Wir werden in diesem Kapitel einige alternative Techniken besprechen, mit denen dieselben Ziele erreicht werden können wie mit den Methoden, die wir in den vorigen Kapiteln diskutiert haben. Diese Techniken sind in Java erst ab Version 8 integriert.

5.1 Ein neuer Blick auf die Themen von Kapitel 4

In Kapitel 4 haben wir uns Techniken angesehen, mit denen Sammlungen von Objekten verarbeitet werden konnten. Der Umgang mit Sammlungen ist eine der fundamentalsten Techniken im Programmieren; fast alle Computerprogramme verarbeiten in irgendeiner Art Sammlungen und wir werden in diesem Buch noch sehr vielen weiteren begegnen. Es ist eine entscheidende Fähigkeit, dies gut zu beherrschen.

Das Programmierenlernen hinsichtlich Sammlungen ist jedoch in den letzten Jahren durch die Einführungen von neuen Programmier-Techniken zu den großen Programmiersprachen komplizierter geworden. Diese Techniken können zwar Ihr *Programm* vereinfachen, aber sie zu *lernen* ist aufwendiger – es gibt jetzt schlicht mehrere unterschiedliche Konstrukte, die zu meistern sind.

Die „neuen“ Techniken sind nicht wirklich neu – sie sind nur neu innerhalb dieser bestimmten Sprachart. Es gab immer unterschiedliche Arten von Sprachen: Java ist beispielsweise eine *imperative objektorientierte* Sprache, und der Stil zur Verarbeitung von Sammlungen, den wir in Kapitel 4 vorgestellt haben, ist eine typische imperative Herangehensweise.

Eine weitere Sprachklasse sind *funktionale Sprachen*; es gibt sie schon eine lange Zeit und sie wenden einen anderen Programmierstil an. Aus Gründen, die wir später noch besprechen werden, hat der funktionale Programmierstil bei der Verarbeitung von Sammlungen in einigen Situationen Vorteile gegenüber dem imperativen Stil. Deshalb haben die Entwickler von Java irgendwann entschieden, einige Elemente der funktionalen Programmierung in Java aufzunehmen. Dies ist in Java Version 8 geschehen (die 2014 veröffentlicht wurde). Die neu hinzugefügten Techniken sind *Streams* (Ströme) und *Lambda-Ausdrücke* (manchmal auch *Closures* genannt). Dies macht Java technisch gesehen zu einer hybriden Sprache: größtenteils imperativ, aber mit einigen funktionalen Elementen.

Für erfahrene Programmierer ist dies eine großartige Sache. Die neuen funktionalen Sprachkonstrukte erlauben es uns, elegantere und ausdrucksstärkere Programme zu schreiben. Auf Anfänger kommt allerdings mehr Arbeit zu: Man muss jetzt zwei unterschiedliche Wege lernen, um ein ähnliches Ziel zu erreichen.

Der neue funktionale Stil, den wir in diesem Kapitel behandeln, wird schnell populär und wird wahrscheinlich innerhalb kurzer Zeit zum vorherrschenden Programmierstil für neuen Code in Java avancieren. Der imperative Stil, der im vorigen Kapitel behandelt wurde, wird jedoch weiterhin für die meisten heutigen Programme verwendet und immer noch als „Standard-Java“ angesehen. Wenn Sie mit Code arbeiten, der von anderen Programmierern stammt (was die meiste Zeit der Fall sein wird), ist es unabdingbar, dass Sie den imperativen Stil verstehen und damit arbeiten können. Beim Programmieren Ihres eigenen Codes können Sie Ihren Stil auswählen: funktionaler und imperativer Stil sind zwei unterschiedliche Wege, um dasselbe Problem zu lösen, und können sich jeweils gegenseitig ersetzen. Von nun an werden wir häufig den funktionalen Stil vorziehen, da er prägnanter und eleganter ist.

Kurz gesagt, ein Anfänger muss sich mit beiden Stilrichtungen der Verarbeitung von Sammlungen vertraut machen, um ein guter Programmierer zu werden. In diesem Buch präsentieren wir immer zuerst den imperativen Stil und führen den funktionalen Stil in den als „fortgeschritten“ markierten Abschnitten (wie diesem Kapitel) ein.

Diese vertiefenden Kapitel können zunächst ausgelassen werden (Sie können also jetzt mit Kapitel 6 fortfahren, wenn Sie möchten) – Sie werden trotzdem Programmieren lernen und weiterhin in der Lage sein, die vorgestellten Aufgaben zu lösen. Diese Abschnitte können auch unabhängig von der chronologischen Reihenfolge gelesen werden (also jetzt überspringen, aber später zurückkehren), wenn Sie gerade wenig Zeit haben und zuerst mit anderen Konstrukten vorankommen möchten. Um einen Gesamteindruck zu erhalten, können Sie diese Abschnitte so lesen, wie sie im Buch erscheinen – auf diese Art lernen Sie die alternativen Konstrukte quasi parallel.

5.2 Monitoring von Tierpopulationen

Wie bisher werden wir ein Beispielprogramm verwenden, um die neuen Konstrukte einzuführen und zu besprechen. Hier wollen wir uns ein System für das Monitoring von Tierpopulationen ansehen.

Ein wichtiges Element von Tierschutz und der Erhaltung gefährdeter Arten ist die Überwachung von Populationsgrößen, um festzustellen, wann der Bestand gesund ist und wann er rückläufig ist. Unser Projekt verarbeitet Sichtungsmeldungen von unterschiedlichen Tierarten, die von Meldern von vielen unterschiedlichen Orten eingesandt werden. Jeder Sichtungsbericht besteht aus dem Namen der Tierart, die gesichtet wurde, der Anzahl der von dieser Art gesichteten Tiere, wer den Bericht sendet (eine ID als ganze Zahl), in welchem Gebiet die Sichtung stattgefunden hat (eine ganze Zahl) und eine Angabe darüber, wann die Sichtung gemacht wurde (ein einfacher numerischer Wert, der zum Beispiel die Anzahl der Tage bezeichnet, die seit Beginn des Experiments vergangen sind).

Die einfachen Datenanforderungen machen es für die Melder an entfernten Orten leicht, verwertbare Informationen in relativ kleinen Einheiten – vielleicht in Form einer Textnachricht – zu einer Basis zu senden, die diese Daten dann sammeln und Berichte erzeugen kann oder die Feldforscher in andere Gebiete ausendet.

Dieses Konzept könnte sowohl bei Projekten eingesetzt werden, die intensiv verwaltet werden – zum Beispiel, wenn in einem Nationalpark Kameras durch Bewegung ausgelöst werden und Angestellte die Kamerabilder auswerten – als auch bei lose strukturierten Crowd-Sourcing-Aktivitäten – wie nationalen Vogelzähltagen, bei denen eine große Gruppe von Freiwilligen Daten mittels Handy-Apps einsendet.

Sie finden eine Teilimplementierung eines solchen Systems unter dem Namen *Tiermonitoring-v1* in den Buchprojekten.

Listing 5.1 zeigt die Klasse **Sichtung**, die wir benutzen, um Details jeder Sichtungsmeldung von einem einzelnen Melder für ein bestimmtes Tier aufzuzeichnen, nachdem die Sichtungsdetails verarbeitet wurden. In diesem Kapitel kümmern wir uns nicht um das Format der eingesandten Daten. Da das Thema dieses Kapitels die fortgeschritteneren Java-Konzepte sind, möchten wir Sie ermutigen, den vollständigen Quelltext des Projekts zu lesen, um herauszufinden, was wir bis jetzt noch nicht ausführlich untersucht haben, und dies für Ihre eigene Entwicklung zu nutzen. Die Klasse **Sichtung** ist jedoch noch relativ geradlinig.

Listing 5.1

Die Klasse `Sichtung`.

```

/**
 * Details der Sichtung einer Tierart, jeweils von einem Melder übermittelt.
 *
 * @author David J. Barnes und Michael Kölling
 * @version 2016.02.29
 */
public class Sichtung
{
    // Das gemeldete Tier.
    private final String tier;
    // Die ID des Melders.
    private final int melder;
    // Anzahl der gesichteten Tiere.
    private final int anzahl;
    // The ID of the area in which they were seen.
    private final int gebiet;
    // The reporting period.
    private final int zeitraum;

    /**
     * Erzeuge einen Datensatz einer Sichtung einer bestimmten Tierart.
     * @param tier das gemeldete Tier
     * @param melder die ID des Melders
     * @param anzahl die Anzahl der gesichteten Tiere (>= 0)
     * @param gebiet die ID des Sichtungsgebiets
     * @param zeitraum der Berichtszeitraum
     */
    public Sichtung(String tier, int melder, int anzahl, int gebiet, int zeitraum)
    {
        this.tier = tier;
        this.melder = melder;
        this.anzahl = anzahl;
        this.gebiet = gebiet;
        this.zeitraum = zeitraum;
    }

    // einige sondierende Methoden hier ausgelassen

    /**
     * Liefere einen String, der die Details zum Tier enthält: gesichtete Anzahl,
     * Sichtungsort, wer die Tiere gemeldet hat, wo und wann sie gesichtet wurden.
     * @return einen String, der die Details zur Sichtung angibt
     */
    public String gibDetails()
    {
        return tier +
            ", Anzahl = " + anzahl +
            ", Gebiet = " + gebiet +
            ", Melder = " + melder +
            ", Zeitraum = " + zeitraum;
    }
}

```

Listing 5.2 zeigt Teile der Klasse `Tiermonitor`, mit deren Hilfe die einzelnen Sichtungen in einer Liste zusammengefasst werden. Zu diesem Zeitpunkt werden alle Sichtungen von allen verschiedenen Meldern und aus allen unterschiedlichen Gebieten in einer einzigen Sammlung gehalten. Unter anderem enthält die Klasse `Sichtung` Methoden zum Ausgeben der Liste, zum Zählen der Gesamtzahl an Sichtungen einer bestimmten Tierart, zum Auflisten aller Sichtungen durch einen bestimmten Melder, zum Entfernen von leeren Datensätzen und so weiter. All diese Methoden wurden mithilfe der grundlegenden Techniken zur Listenverarbeitung aus Kapitel 4 implementiert: Iteration über der gesamten Liste; Verarbeitung ausgewählter Listenelemente aufgrund bestimmter Bedingungen (Filtern); sowie das Entfernen von Elementen. Die hier implementierten Methoden sind für eine Anwendung dieser Art nicht vollständig, wir haben sie hier aber implementiert, um die unterschiedlichen Arten der Listenverarbeitung zu demonstrieren.

```

import java.util.ArrayList;
import java.util.Iterator;

/**
 * Monitor-Auszählungen von verschiedenen Tierarten.
 * Sichtungen werden von Meldern aufgezeichnet.
 *
 * @author David J. Barnes und Michael Kölling
 * @version 2016.02.29 (imperative Version)
 */
public class Tiermonitor
{
    // Datensätze aller Tiersichtungen.
    private ArrayList<Sichtung> sichtungen;

    /**
     * Erzeuge einen Tiermonitor.
     */
    public Tiermonitor()
    {
        this.sichtungen = new ArrayList<>();
    }

    /**
     * Füge zur aktuellen Liste die Sichtungen aus der angegebenen Datei hinzu.
     * @param dateiname eine CSV-Datei mit den Sichtsungsdatensätzen
     */
    public void sichtungenHinzufuegen(String dateiname)
    {
        SichtungLeser leser = new SichtungLeser();
        sichtungen.alleHinzufuegen(leser.gibSichtungen(dateiname));
    }

    /**
     * Gib die Details aller Sichtungen aus.
     */
    public void listeAusgeben()
    {
        for(Sichtung datensatz : sichtungen) {
            System.out.println(datensatz.gibDetails());
        }
    }

    // weitere Methoden hier ausgelassen
}

```

Listing 5.2
Die Klasse
Tiermonitor.

Bisher verwendet der Quelltext die imperativen Techniken zur Verarbeitung von Sammlungen, die in Kapitel 4 eingeführt wurden. Wir wollen dies als Basis nutzen, um schrittweise den Code zur Listenverarbeitung durch funktionale Konstrukte zu ersetzen.

Übung 5.1 Öffnen Sie das Projekt *Tiermonitoring-v1* und erzeugen Sie eine Instanz der Klasse **Tiermonitor**. Die Datei *sichtungen.csv* im Projektordner enthält eine kleine Sammlung von Sichtsungsdatensätzen im CSV-Format (*Comma-Separated Values*). Übergeben Sie den Namen dieser Datei (also „sichtungen.csv“) der Methode **sichtungenHinzufuegen** im **Tiermonitor**-Objekt und rufen Sie dann **listeAusgeben** auf, um Details der eingelesenen Daten anzuzeigen.

Übung 5.2 Probieren Sie die anderen Methoden des **Tiermonitor**-Objekts aus, benutzen Sie dazu Tiernamen, Melder- und Gebiet-IDs, die in der Ausgabe von **listeAusgeben** gezeigt sind.

Übung 5.3 Lesen Sie sich den vollständigen Quelltext der Klasse `Tiermonitor` durch, um zu verstehen, wie die Methoden arbeiten. Die Klasse verwendet nur Techniken, die wir in den vorigen Kapiteln behandelt haben, Sie sollten daher in der Lage sein, die Details herauszufinden.

Übung 5.4 Warum verwendet die Methode `entferneNullAnzahl` eine `while`-Schleife mit einem Iterator anstatt einer `for-each`-Schleife wie in den anderen Methoden? Könnte die Methode auch mit einer `for-each`-Schleife geschrieben werden? Könnten die anderen Methoden mit einer `while`-Schleife funktionieren?

Konzept

Ein **Lambda-Ausdruck** ist ein Codeabschnitt, der für eine spätere Ausführung gespeichert werden kann.

5.3 Lambda-Ausdrücke – ein erster Blick

Im Zentrum des neuen funktionalen Programmierstils zur Verarbeitung von Sammlungen steht das Konzept der *Lambda-Ausdrücke*.

Die Grundidee besteht darin, einen *Codeabschnitt* als Parameter an eine Methode zu übergeben, die dieses Codestück später ausführen kann, wenn es benötigt wird.

Bis jetzt haben wir verschiedene Parametertypen kennengelernt – ganze Zahlen, Zeichenketten, Objekte –, doch diese waren allesamt Bestandteile von *Daten*, nicht von *Code*. Die Möglichkeit, Code als Parameter zu übergeben, ist mit Java 8 neu hinzugekommen, und erlaubt uns, einige sehr nützliche Dinge zu tun.

5.3.1 Verarbeitung einer Sammlung mit einem Lambda-Ausdruck

Als wir unsere Sammlungen in Kapitel 4 verarbeitet haben, haben wir eine Schleife verwendet, um die Elemente der Sammlung eines nach dem anderen zu holen, und daraufhin haben wir etwas mit jedem Element gemacht. Die Struktur ähnelt dabei immer dem folgenden Pseudocode:

```
loop (für jedes Element in der Sammlung):
    nimm ein Element;
    mache etwas mit dem Element;
ende loop
```

Es ist egal, ob wir eine `for`-Schleife, eine `for-each`-Schleife oder eine `while`-Schleife verwenden oder ob wir einen Iterator oder einen Index benutzen, um auf die Elemente zuzugreifen – das Prinzip ist dasselbe.

Beim Einsatz eines Lambda-Ausdrucks wird das Problem anders angegangen. Wir übergeben ein paar Codezeilen an die Sammlung mit der Anweisung „Tue dies mit jedem Element in der Sammlung“. Die Codestruktur sieht ungefähr so aus:

```
sammlung.tueDiesMitJedemElement(ein wenig Code);
```

Offensichtlich sieht der Code viel einfacher aus. Am wichtigsten ist jedoch: die Schleife ist ganz verschwunden. Tatsächlich ist sie nicht weg – sie ist lediglich in die `tueDiesMitJedemElement`-Methode verschoben worden –, aber sie ist aus unserem eigenen Code verschwunden; wir müssen sie nicht mehr jedes Mal schreiben.

Die Sammlungen sind nun leistungsstärker: Anstatt nur ihre Elemente an uns aushändigen zu können, damit wir diese irgendwie bearbeiten, ist die Sammlung jetzt in der Lage, die Elemente für uns zu bearbeiten. Dies erleichtert uns das Leben.

Wir können uns das folgendermaßen vorstellen: Angenommen, Sie müssen jedem Kind in einer Schulklasse die Haare schneiden. Wenn Sie die Aufgabe nach der alten Art angehen, dann sagen Sie zu dem Lehrer: „Schicken Sie mir das erste Kind.“ Dann schneiden Sie dem Kind die Haare. Danach sagen Sie: „Schicken Sie mir das nächste Kind.“ Eine weitere Haarschneidesitzung folgt. Und so geht es weiter, bis alle Kinder ihren Haarschnitt bekommen haben. Dies ist die Schleife im alten Stil von „nächstes Element-Verarbeitung“.

Beim neuen Stil machen Sie Folgendes: Sie schreiben Anweisungen auf, wie man Haare schneidet, und dann geben Sie dies dem Lehrer und sagen: „Führen Sie dies bei jedem Kind in Ihrer Klasse durch.“ (Der Lehrer repräsentiert hier die Sammlung.) Tatsächlich ist ein interessanter Nebeneffekt dieses Ansatzes, dass wir noch nicht einmal wissen müssen, wie der Lehrer diese Aufgabe bewältigen wird. Zum Beispiel könnte er, anstatt die Haare selbst zu schneiden, sich entscheiden, für jedes einzelne Kind eine andere Person zu beauftragen, sodass alle Haare gleichzeitig geschnitten werden. Der Lehrer würde also einfach die Anweisungen weiterreichen, die Sie ihm gegeben haben.

Ihr Leben ist plötzlich viel einfacher. Der Lehrer übernimmt den Großteil der Arbeit für Sie. Dies ist genau das Prinzip, wie die neuen Sammlungen in Java 8 für Sie arbeiten.

Konzept

Beim **funktionalen Programmierstil zur Verarbeitung von Sammlungen** holen wir nicht jedes Element einzeln, stattdessen übergeben wir ein Codesegment an die Sammlung, das auf jedes Element angewandt werden soll.

5.3.2 Grundlegende Syntax eines Lambda-Ausdrucks

Ein Lambda-Ausdruck ähnelt in gewisser Weise einer Methode: es ist ein Codeabschnitt, der für eine bestimmte Aufgabe definiert ist, der aber nicht unmittelbar ausgeführt wird. Anders als eine Methode jedoch gehört ein Lambda-Ausdruck nicht zu einer Klasse und wir werden ihn nicht benennen. Es ist hilfreich, die Syntax eines Lambda-Ausdrucks mit der einer Methode zu vergleichen, die dieselbe Aufgabe ausführt.

Eine Methode, um die Details einer Sichtung auszugeben, könnte folgendermaßen aussehen:

```
public void sichtungAusgeben(Sichtung datensatz)
{
    System.out.println(datensatz.gibDetails());
}
```

Der entsprechende Lambda-Ausdruck dazu lautet:

```
(Sichtung datensatz) ->
{
    System.out.println(datensatz.gibDetails());
}
```

Sie können jetzt leicht die Unterschiede und die Gemeinsamkeiten sehen. Die Unterschiede sind:

- Es gibt kein **public**- oder **private**-Schlüsselwort – die Sichtbarkeit wird als **public** vorausgesetzt.
- Es wird kein Rückgabewert angegeben. Das liegt daran, dass der Compiler dies aus der letzten Anweisung im Rumpf des Lambda-Ausdrucks herausfinden kann. In diesem Fall ist die **println**-Methode **void**, das heißt der Rückgabetypp des Lambda-Ausdrucks ist **void**.
- Der Lambda-Ausdruck ist nicht benannt; er beginnt mit der Parameterliste.
- Ein Pfeil (->) trennt die Parameterliste vom Rumpf des Lambda-Ausdrucks – dies ist die unverwechselbare Schreibweise für einen Lambda-Ausdruck.

Sie werden feststellen, dass Lambda-Ausdrücke oft für relativ einfache, einmalig auszuführende Aufgaben eingesetzt werden, die nicht die Komplexität einer ganzen Klasse benötigen. Einige Informationen zu einem Lambda-Ausdruck werden standardmäßig geliefert oder aus dem Kontext erschlossen – seine Sichtbarkeit und der Rückgabetypp – und wir haben keinen Einfluss darauf. Und da ein Lambda-Ausdruck keine ihm zugeordnete Klasse hat, gibt es außerdem keine Instanzfelder und keinen Konstruktor. Dies führt zu einem sehr spezialisierten Einsatz der Lambda-Ausdrücke. Aufgrund des fehlenden Namens sind Lambda-Ausdrücke auch als *anonyme Funktionen* bekannt.

Wir werden den Lambda-Ausdrücken und Funktionsparametern in diesem Buch noch später mehrmals begegnen, zum Beispiel wenn wir *funktionale Interfaces* in Kapitel 12 einführen und wenn wir grafische Benutzerschnittstellen (GUIs) in Kapitel 13 schreiben. Für den Moment werden wir jedoch untersuchen, wie Lambda-Ausdrücke mit unseren Sammlungen eingesetzt werden.

5.4 Die forEach-Methode von Sammlungen

Als wir oben das Konzept der Lambda-Ausdrücke eingeführt haben, haben wir die Idee einer **tueDiesMitJedemElement**-Methode in der Sammlungsklasse angeschnitten. Diese Rolle nimmt in Java die **forEach**-Methode ein.

Wenn wir eine Sammlung namens **meineListe** haben, können wir schreiben:

```
meineListe.forEach( ... Code, der auf jedes Listenelement
                    angewandt wird ... )
```

Der Parameter für diese Methode wird ein Lambda-Ausdruck – ein Codesegment – mit der oben gezeigten Syntax sein. Die **forEach**-Methode führt dann den Lambda-

Ausdruck für jedes der obigen Listenelemente aus, wobei der Reihe nach jedes Listenelement als Parameter dem Lambda-Ausdruck übergeben wird.

Nun wollen wir uns den richtigen Code ansehen. Die Methode `listeAusgeben` aus unserer `Tiermonitor`-Klasse sieht folgendermaßen aus:

```
/**
 * Gib die Details aller Sichtungen aus.
 */
public void listeAusgeben()
{
    for(Sichtung datensatz : sichtungen) {
        System.out.println(datensatz.gibDetails());
    }
}
```

Wir können jetzt die folgende Version dieser Methode schreiben, die auf Lambda-Ausdrücken basiert:

```
/**
 * Gib die Details aller Sichtungen aus.
 */
public void listeAusgeben()
{
    sichtungen.forEach(
        (Sichtung datensatz) ->
        {
            System.out.println(datensatz.gibDetails());
        }
    );
}
```

Der Rumpf dieser neuen Version von `listeAusgeben` besteht aus einer einzigen Anweisung (selbst wenn sich diese über sechs Zeilen hinzieht), welche ein Aufruf an die `forEach`-Methode der `sichtungen`-Liste ist. Zugegebenermaßen sieht es jetzt noch nicht einfacher als vorher aus, aber wir werden gleich dort hinkommen. Es ist wichtig zu bemerken, dass es in dieser Version, anders als in der vorigen, keine explizite Schleifenanweisung gibt – die Iteration wird *implizit* von der `forEach`-Methode durchgeführt.

Der Parameter der `forEach`-Methode ist ein Lambda-Ausdruck. Der Lambda-Ausdruck selbst hat einen Parameter namens `datensatz`. Jedes Element der `sichtungen`-Liste wird der Reihe nach als Parameter für diesen Lambda-Ausdruck verwendet und der Rumpf des Lambda-Ausdrucks wird ausgeführt. So werden die Details von jedem Element ausgegeben.

Dieser syntaktische Stil unterscheidet sich völlig von allem, was Sie in den vorigen Kapiteln dieses Buches kennengelernt haben. Nehmen Sie sich daher unbedingt genügend Zeit, um dieses erste Beispiel eines Lambda-Ausdrucks in Aktion zu verstehen. Wir werden in Kürze einige leichte Veränderungen an der hier verwendeten Syntax zeigen, doch dieses Beispiel illustriert den typischen Einsatz eines Lambda-Ausdrucks und die grundlegenden Ideen finden sich in allen kommenden Beispielen wieder:

- Ein Lambda-Ausdruck beginnt mit einer Parameterliste.
- Ein Pfeilsymbol folgt auf die Parameter.
- Auszuführende Anweisungen kommen nach dem Pfeilsymbol.

Übung 5.5 Öffnen Sie das Projekt *Tiermonitoring-v1* und schreiben Sie die Methode `ListeAusgeben` in Ihrer Version der *Tiermonitor*-Klasse um, sodass nun ein Lambda-Ausdruck verwendet wird, wie oben gezeigt. Testen Sie, ob die Methode noch funktioniert und dasselbe Ergebnis wie bisher liefert.

5.4.1 Variationen der Syntax von Lambda-Ausdrücken

Eines der Ziele der Lambda-Syntax in Java ist es, dass Code klar und präzise geschrieben werden kann. Die Möglichkeit, den Zugriffsmodifikator `public` und den Rückgabtyp auszulassen, stellt den Auftakt dazu dar. Der Compiler erlaubt es uns jedoch, noch darüber hinauszugehen. Es können mehrere weitere Elemente ausgelassen werden, wenn der Compiler diese selbst herausfinden kann.

Das erste Element ist der Parametertyp für den Lambda-Ausdruck (in unserem Fall **datensatz**). Da der Typ des Lambda-Parameters immer mit dem Typ der Sammlungselemente übereinstimmen muss, kann der Compiler diesen Typ herausfinden, und wir können ihn auslassen. Unser Code sieht dann folgendermaßen aus:

```
sichtungen.forEach(
    (datensatz) ->
    {
        System.out.println(datensatz.gibDetails());
    }
);
```

Die nächste Vereinfachung betrifft die Klammern, sowohl die runden als auch die geschweiften:

- Wenn die Parameterliste nur einen einzigen Parameter enthält, dann können die Klammern weggelassen werden.
- Enthält der Rumpf des Lambda-Ausdrucks nur eine einzige Anweisung, dann können die geschweiften Klammern weggelassen werden.

Durch die Anwendung dieser Vereinfachungen auf unseren Code wird dieser nun so kurz, dass er in eine einzige Zeile passt, und wir können den **forEach**-Aufruf nun folgendermaßen schreiben:

```
sichtungen.forEach(datensatz) -> System.out.println(datensatz.gibDetails());
```

Dies ist die Version, die wir ab nun in unserem Quelltext bevorzugt verwenden.

Auch an dieser Stelle möchten wir Sie noch einmal anregen sicherzustellen, dass Sie verstanden haben, wie diese reduzierte Syntax der Beschreibung entspricht, da in der Praxis Lambda-Ausdrücke oft in dieser Form geschrieben werden.

Das Projekt *Tiermonitoring-v2* ist im funktionalen Stil wie hier dargestellt geschrieben worden und kann verwendet werden, falls Sie Ihre Lösungen von einigen der Aufgaben überprüfen möchten. Machen Sie trotzdem die Übungen erst einmal selbst und verwenden Sie dieses Projekt nur, wenn Sie wirklich nicht weiterkommen.

Vorteile von Lambda-Ausdrücken

Wenn wir Lambda-Ausdrücke in der Praxis verwenden, ist der Lambda-Code häufig kurz und einfach und wir können die eben eingeführte abkürzende Schreibweise benutzen. Dadurch wird unser Quelltext kurz und prägnant und ist einfach zu schreiben. Hinzu kommt: Da wir die Schleife nicht mehr selbst programmieren, machen wir wahrscheinlich auch weniger Fehler – wenn wir die Schleife nicht schreiben, können wir sie nicht falsch machen.

Es gibt jedoch noch weitere Vorteile. Einer davon ist die Möglichkeit, Streams zu verwenden, die im nächsten Abschnitt besprochen werden. Ein weiterer Vorteil ist die Nebenläufigkeit (auch als parallele Programmierung bezeichnet).

Die meisten Rechner heute haben mehrere Kerne und sind grundsätzlich in der Lage, parallel zu arbeiten. Es jedoch extrem schwierig, Code zu schreiben, der mehrere Kerne effizient nutzt. Ein Großteil unserer Programme verwendet meistens nur einen Kern, der Rest der verfügbaren Prozessorleistung wird verschwendet. Code zu programmieren, der alle Kerne verwendet, ist ein sehr fortgeschrittenes Thema, und nur wenige Programmierer kommen jemals so weit.

Mithilfe der Lambda-Ausdrücke und durch das Verlagern der Schleife in die Sammlung können wir nun potenziell Sammlungen dazu benutzen, die parallele Verarbeitung für uns zu übernehmen. Möchten wir also Parallelität haben, so müssen wir nur eine entsprechende Sammlung verwenden und der ganze schwer zu programmierende Code für die parallele Verarbeitung wird jetzt innerhalb der Sammlungsklasse versteckt und das Schreiben des Quelltextes dafür wird uns abgenommen. Unser Code sieht kurz und einfach aus. Plötzlich wird es – mit nur sehr wenig zusätzlichem Aufwand – möglich, alle Kerne in Ihrem Laptop zu benutzen.

Wir werden in diesem Buch parallele Programmierung nicht weiter besprechen. Es ist jedoch interessant zu wissen, dass Sie gut darauf vorbereitet sind, später nebenläufige Sammlungsklassen zu verwenden, wenn Sie den Umgang mit Lambda-Ausdrücken und Streams gut beherrschen.

5.5 Streams

Die `forEach`-Methode der Klasse `ArrayList` (und verwandten Sammlungsklassen) ist nur ein Beispiel für eine Methode, die *Streams* (Ströme) verwendet, welche mit Java 8 eingeführt wurden. Streams sind ein bisschen wie Sammlungen, doch mit einigen wichtigen Unterschieden:

- Auf die Elemente in einem Stream wird nicht über einen Index zugegriffen, sondern in der Regel sequenziell.
- Die Inhalte und die Reihenfolge der Elemente des Streams können nicht verändert werden – für Änderungen muss ein neuer Stream erzeugt werden.
- Ein Stream kann eventuell unendlich sein!

Konzept

Streams vereinen die Verarbeitung von Elementen einer Sammlung und anderer Datenmengen. Ein Stream stellt nützliche Methoden zur Verfügung, um diese Datenmengen zu manipulieren.

Das Konzept der Streams wird eingesetzt, um die Verarbeitung von Datenmengen zu vereinheitlichen, unabhängig davon, woher diese Daten stammen. Es könnte die Verarbeitung von Elementen aus einer Sammlung sein (wie wir es besprochen haben), von Nachrichten, die über ein Netzwerk hereinkommen, von Textzeilen aus einer Textdatei oder von allen Zeichen einer Zeichenkette. Es ist egal, aus welcher Quelle die Daten kommen – wir können sie als einen Datenstrom ansehen und dann darauf dieselben Techniken und Methoden anwenden, um die jeweiligen Elemente zu verarbeiten.

Mit den Streams kommt in Java eine weitere wichtige neue Funktion gegenüber vorherigen Möglichkeiten zur Verarbeitung von Sammlungen hinzu: die Fähigkeit zur sicheren parallelen Verarbeitung. Es gibt viele Situationen, in denen die Reihenfolge, in der die Elemente einer Sammlung verarbeitet werden, keine Rolle spielt oder in denen die Verarbeitung eines Elements aus der Sammlung im Wesentlichen unabhängig von den anderen Elementen ist. Treffen diese Bedingungen zu, so kann die Parallelisierung die Sammlungsverarbeitung bedeutend beschleunigen, wenn es um große Datenmengen geht. Wir werden dies hier jedoch nicht weiter besprechen.

Eine **ArrayList** ist selbst kein Stream, aber wir können die **ArrayList** auffordern, die Quelle eines Streams zur Verfügung zu stellen, indem wir ihre **stream**-Methode aufrufen. Der zurückgelieferte Stream wird alle Elemente der **ArrayList** in Indexreihenfolge enthalten. Von der Parallelisierung einmal abgesehen, bieten Streams uns eigentlich nichts, was wir nicht bereits vorher mit den bekannten Sprachmerkmalen programmieren konnten. Doch Streams lösen elegant viele der am häufigsten vorkommenden Aufgaben, die man auf Sammlungen durchführen kann, und zwar in einer Form, die sowohl viel leichter zu programmieren als auch einfacher zu verstehen ist, als wir es bisher gewohnt sind. Insbesondere werden wir sehen, dass man durch den Einsatz von Streams keine expliziten Schleifen zur Verarbeitung einer Sammlung schreiben muss – was wir jetzt mit einer Neufassung der **listeAusgeben**-Methode in **Übung 5.6** durchführen werden.

Übung 5.6 Modifizieren Sie Ihre eigene **listeAusgeben**-Methode, wie im Text besprochen: Probieren Sie alle Syntaxvariationen für Lambda-Ausdrücke aus, die wir vorgestellt haben. Kompilieren Sie Ihren Code und testen Sie das Projekt mit jeder Variation, um zu sicherzustellen, dass Ihre Syntax korrekt ist.

Wir haben bereits einiges davon mit der **forEach**-Methode gesehen; im Rest dieses Kapitels werden wir die Methoden **filter**, **map** und **reduce** näher untersuchen, die auf Streams definiert sind.

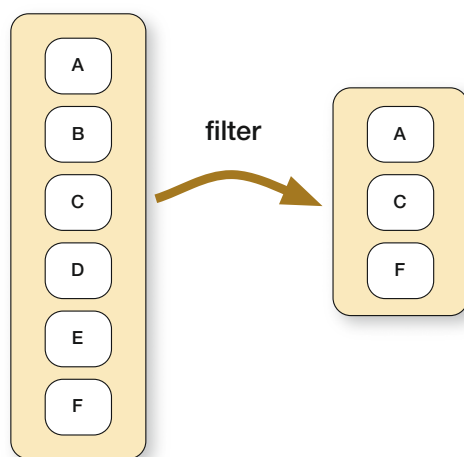
5.5.1 Filter-Map-Reduce

Wir haben oben gesagt, dass die Inhalte eines Streams nicht verändert werden können, bei Änderungen muss also ein neuer Stream erzeugt werden. Mit nur drei Funktionstypen kann eine Vielzahl unterschiedlicher Verarbeitungsarten realisiert werden: *filter*, *map* und *reduce*. Diese drei können variiert und kombiniert werden, und damit können wir fast alles erreichen, was wir jemals vorhaben.

Bevor wir uns die konkreten Java-Methoden ansehen, wollen wir uns noch ein wenig mit dem Grundgedanken aufhalten, der diesen Funktionen zugrunde liegt. Alle werden auf einen Stream angewandt, um ihn auf eine bestimmte Weise zu verarbeiten. Und sie sind alle sehr verbreitet und nützlich.

Die Funktion filter

Die Funktion *filter* erhält einen Stream, wählt einige seiner Elemente aus und erzeugt einen neuen Stream, der nur aus den ausgewählten Elementen besteht (Abbildung 5.1). Einige Elemente wurden *herausgefiltert*. Das Ergebnis ist ein Stream mit weniger Elementen (eine Teilmenge des ursprünglichen Streams).



Konzept

Wir können die Funktion **filter** auf einen Stream anwenden, um nur bestimmte Elemente auszuwählen.

Abbildung 5.1

Anwendung der Funktion *filter* auf einen Stream.

Ein Beispiel aus unserer Sammlung der Tiersichtungen wäre es, nur die Sichtungen von Elefanten aus allen aufgezeichneten Sichtungen zu selektieren. Wir beginnen mit allen Sichtungen, wenden einen Filter an, der nur Elefanten auswählt, und erhalten einen Stream aller Elefantensichtungen.

Die Funktion map

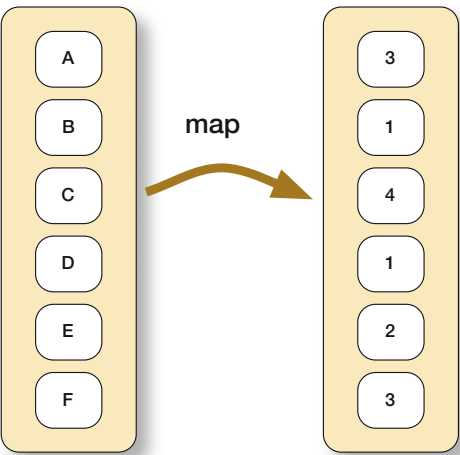
Die Funktion *map* erhält einen Stream und erzeugt einen neuen Stream, wobei jedes Element des ursprünglichen Streams auf ein neues anderes Element im neuen Stream abgebildet wird (Abbildung 5.2). Der neue Stream hat genauso viele Elemente wie der alte, doch Typ und Inhalt jedes Elements können sich unterscheiden – das neue Element wurde irgendwie vom ursprünglichen Element abgeleitet.

Konzept

Wir können die Funktion **map** anwenden, um einen Stream auf einen neuen Stream abzubilden, wobei jedes Element des ursprünglichen Streams durch ein neues Element ersetzt wird, das vom Original abgeleitet wurde.

Abbildung 5.2

Anwendung der Funktion *map* auf einen Stream.



Ein Beispiel aus unserem Projekt wäre, jedes gesichtete Objekt aus dem ursprünglichen Stream mit der Anzahl der in dieser Sichtung entdeckten Tiere zu ersetzen. Wir beginnen mit einem Stream der gesichteten Objekte und haben am Ende einen Stream von ganzen Zahlen.

Die Funktion *reduce*

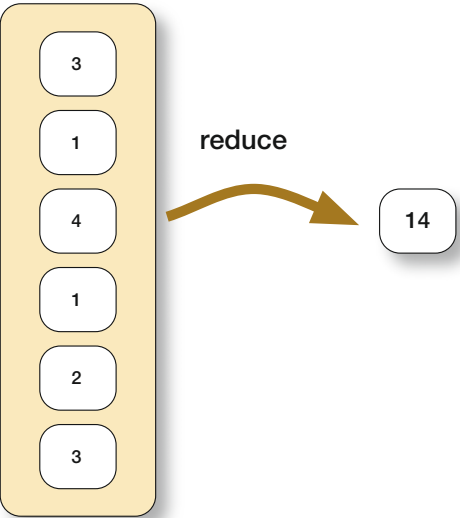
Die Funktion *reduce* erhält einen Stream und reduziert alle Elemente auf ein einziges Ergebnis (Abbildung 5.3). Dies kann auf verschiedene Arten erreicht werden, zum Beispiel indem alle Elemente addiert werden oder indem einfach das kleinste Element aus dem Stream ausgewählt wird. Wir beginnen mit einem Stream und haben am Ende einen einzelnen Ergebniswert.

Konzept

Wir können die Funktion **reduce** auf einen Stream anwenden, um den Stream zu reduzieren. Das heißt, die Funktion erhält den gesamten Stream als Eingabe und gibt einen einzigen Ergebniswert zurück.

Abbildung 5.3

Anwendung der Funktion *reduce* auf einen Stream.

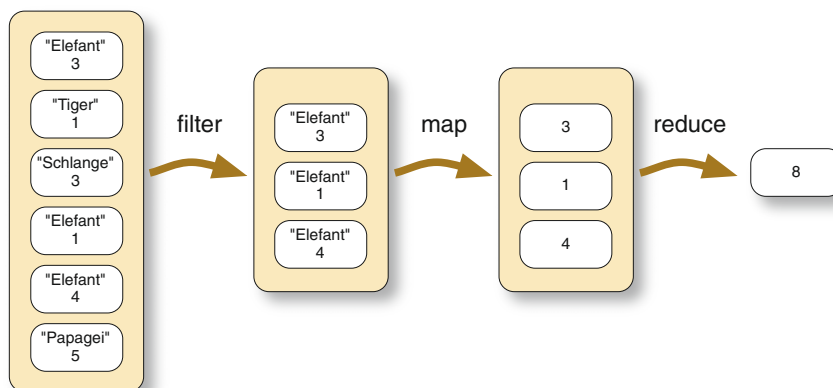


In unserem Tiermonitoring-Projekt könnte dies beispielsweise nützlich sein, wenn wir zählen möchten, wie viele Elefanten wir gesehen haben: Sobald wir einen Stream mit Elefantensichtungen haben, können wir *reduce* verwenden, um diese zu addieren. (Zur Erinnerung: jede Instanz einer Sichtung kann aus mehreren Tierarten bestehen.)

5.5.2 Pipelines

Streams und Stream-Funktionen werden sogar noch nützlicher, wenn mehrere Funktionen zu einer *Pipeline* zusammengefasst werden. Eine Pipeline ist die Verkettung von zwei oder mehreren dieser Funktionen, sodass eine nach der anderen angewandt werden kann.

Wir wollen annehmen, dass wir herausfinden möchten, wie viele Elefantensichtungen wir aufgezeichnet haben. Wir können dazu die Funktionen *filter*, *map* und *reduce* hintereinander anwenden (Abbildung 5.4).



Konzept

Eine **Pipeline** ist die Kombination von zwei oder mehr Stream-Funktionen in einer Kette, wobei die einzelnen Funktionen nacheinander angewandt werden.

Abbildung 5.4

Eine Pipeline von Stream-Funktionen.

In diesem Beispiel wenden wir zuerst die *filter*-Funktion auf alle Sichtungen an, um nur die Elefantensichtungen auszuwählen, dann bilden wir mit der *map*-Funktion jedes Sichtsungsobjekt auf die Anzahl der beobachteten Elefanten dieser Sichtung ab und zum Schluss addieren wir diese Werte innerhalb einer *reduce*-Funktion. Beachten Sie, dass wir es nach der Anwendung der *map*-Funktion nicht mehr mit einem Stream von **Sichtung**-Objekten zu tun haben, sondern mit einem Stream von ganzzahligen Werten. Diese Form der Transformation zwischen Eingabe-Stream und Ausgabe-Stream kommt bei der *map*-Funktion sehr häufig vor.

In Pseudocode können wir dies folgendermaßen schreiben:

```
sichtungen.filter(name == elefant).map(anzahl).reduce(addieren);
```

Dies ist keine korrekte Java-Syntax, aber Sie bekommen eine Vorstellung von der Idee. Wir werden uns weiter unten den korrekten Java-Code ansehen.

Durch die verschiedenen Kombinationen dieser Funktionen kann man viele unterschiedliche Operationen bekommen. Wenn wir beispielsweise wissen möchten, wie viele Berichte von einem bestimmten Melder an einem bestimmten Tag eingereicht wurden, können wir schreiben:

```
sichtungen.filter(melder == melderID)
               .filter(zeitraum == tagID)
               .reduce(anzahl elemente);
```

Pipelines beginnen immer mit einer *Quelle* (einem Stream), gefolgt von einer Reihe von Operationen. Es gibt zwei Arten von Operationen: *intermediäre Operationen* und *terminale Operationen*. Die letzte Operation einer Pipeline muss immer eine terminale Operation sein und alle anderen müssen intermediäre Operationen sein:

```
quelle.inter1(...).inter2(...).inter3(...).terminal(...);
```

Übung 5.7 Schreiben Sie Pseudocode, um festzustellen, wie viele Elefanten ein bestimmter Melder (**melderID**) an einem bestimmten Tag (**tagID**) gesehen hat.

Übung 5.8 Schreiben Sie Pseudocode, um einen Stream zu erzeugen, der nur Sichtungen enthält, deren **anzahl** größer als 0 ist.

Übung 5.9 Öffnen Sie noch einmal das Projekt *Musiksammlung* aus Kapitel 4 und nehmen Sie an, dass jedes **Track**-Objekt der Sammlung eine Variable für die Anzahl enthält, wie oft der Track abgespielt wurde. Schreiben Sie Pseudocode, um festzustellen, wie oft die Tracks des Künstlers Bill Broonzy insgesamt abgespielt wurden.

Intermediäre Operationen erzeugen immer einen neuen Stream, auf den die nächste Operation angewandt werden kann, während terminale Operationen einen Ergebniswert liefern oder leer sind (wie **forEach**, das eine terminale Operation ist). Die Dokumentation der Stream-Methode gibt Ihnen immer Auskunft darüber, ob eine Operation intermediär oder terminal ist.

Wir werden im Folgenden die Java-Syntax der Methoden vorstellen, die diese Funktionen implementieren. Hier treffen Streams und Lambda-Ausdrücke aufeinander: Der Parameter für diese Methoden ist ein Lambda-Ausdruck.

5.5.3 Die Methode filter

Wie bereits angesprochen, erzeugt die Methode **filter** eine Teilmenge des ursprünglichen Streams. In der Regel wird diese Teilmenge aus denjenigen Elementen bestehen, die einer Bedingung genügen; das heißt, diese Elemente haben irgendetwas, was wir bewahren möchten, sodass wir mehr mit diesen bestimmten Elementen als mit den anderen weiterarbeiten wollen. Bei dem *Musiksammlung*-Projekt möchten wir vielleicht alle Tracks eines bestimmten Künstlers finden. Beim *Tiermonitoring*-Projekt möchten wir eventuell alle Sichtungen eines bestimmten Tiers finden. Bei beiden wird über die jeweilige Liste iteriert und für jedes Element getestet, um festzustellen, ob es einer bestimmten Anforderung entspricht. Falls die geforderte Bedingung erfüllt wird, dann verarbeiten wir das Element auf irgendeine Weise weiter.

Die **filter**-Methode eines Streams ist eine intermediäre Operation, die an ihren Ausgabe-Stream nur diejenigen Elemente des Eingabe-Streams weitergibt, die eine

gegebene Bedingung erfüllen. Die Methode bekommt einen Lambda-Ausdruck, der ein Element erhält, und gibt einen **boolean**-Wert als Ergebnis zurück. (Ein Lambda-Ausdruck, der ein Element entgegennimmt und **true** oder **false** zurückgibt, wird *Prädikat* genannt.) Ist das Ergebnis **true**, dann wird das Element in den Ausgabe-Stream aufgenommen, andernfalls nicht. Wenn wir beispielsweise nur diejenigen **Sichtung**-Objekte selektieren möchten, die sich auf Elefanten beziehen, dann könnte der Code dazu so aussehen:

```
sichtungen.stream()
    .filter(s -> "Elefant".equals(s.gibTier()))
```

Beachten Sie, dass wir zuerst einen Stream aus der Liste der Sichtungen erzeugen müssen, um die *filter*-Funktion anwenden zu können. Es ist nicht nötig, den Parametertyp des Lambda-Ausdrucks anzugeben, da der Compiler weiß, dass **ArrayList<Sichtung>** einen Stream von **Sichtung**-Objekten liefern wird, und somit haben die Parameter des Prädikats den Typ **Sichtung**.

Allgemeiner gesagt ist es sinnvoll, den Vorgang des Filterns zum Teil einer Methode zu machen, die unterschiedliche Tierarten selektieren könnte. Wir können daher die Methode **sichtungenAusgebenUeber** unseres ursprünglichen Projekts wie folgt als Stream-basierte Version neu formulieren:

```
/**
 * Gib die Details aller Sichtungen des angegebenen Tiers aus.
 * @param tier die Tierart
 */
public void sichtungenAusgebenUeber(String tier)
{
    sichtungen.stream()
        .filter(s -> tier.equals(s.gibTier()))
        .forEach(s -> System.out.println(s.getDetails()));
}
```

Es ist wichtig zu wissen, dass jede Stream-Operation ihren Eingabe-Stream unverändert beibehält. Jede Operation erzeugt – basierend auf seiner Eingabe und der ausgeführten Operation – einen neuen Stream, der an die nächste Operation in der Folge weitergereicht wird. In diesem Fall ist die terminale Operation eine **forEach**-Methode, die als Ergebnis die Sichtungen liefert, die den vorhergehenden Filterprozess „überlebt“ haben. Die **forEach**-Methode ist eine terminale Operation, weil sie keinen weiteren Stream als Ergebnis hat – es ist eine **void**-Methode.

Übung 5.10 Schreiben Sie die Methode **sichtungenAusgebenUeber** in Ihrer **Tiermonitor**-Klasse neu, sodass sie nun Streams und Lambda-Ausdrücke verwendet, wie wir es oben gezeigt haben. Testen Sie die Klasse, um sicherzustellen, dass das Projekt immer noch wie vorher funktioniert.

Übung 5.11 Schreiben Sie eine Methode in der **Tiermonitor**-Klasse, um Details aller aufgezeichneten Sichtungen eines bestimmten Tags (**tagID**) auszugeben, der als Parameter der Methode übergeben wird.

Übung 5.12 Schreiben Sie eine Methode, die zwei **filter**-Aufrufe verwendet, um Details aller Sichtungen einer bestimmten Tierart an einem bestimmten Tag auszugeben – die Methode bekommt den Tiernamen und die Tages-ID als Parameter.

Übung 5.13 Hat die Reihenfolge der beiden **filter**-Aufrufe in Ihrer Lösung der vorigen Aufgabe einen Einfluss? Begründen Sie Ihre Antwort.

5.5.4 Die Methode **map**

Die Methode **map** ist eine intermediäre Operation, die einen Stream mit unterschiedlichen Objekten erzeugt, die möglicherweise auch unterschiedlichen Typs sind, indem jeder der ursprünglichen Elemente auf ein neues Element im neuen Stream abgebildet wird.

Dies wollen wir wieder am Beispiel illustrieren: Wenn wir die Details einer Sichtung ausgeben, sind wir eigentlich nur an der Zeichenkette interessiert, die von der Methode **gibDetails** zurückgeliefert wird, und nicht am **Sichtung**-Objekt selbst. Eine Alternative zum obigen Code (wo wir **gibDetails()** innerhalb von **System.out.println** aufgerufen haben) wäre, das **Sichtung**-Objekt zuerst auf den String abzubilden, der von **gibDetails** kommt. Die terminale Operation kann dann einfach diese Details ausgeben:

```
/**
 * Gib alle Sichtungen des angegebenen Melders aus.
 * @param melder die ID des Melders
 */
public void sichtungenAusgebenVon(int melder)
{
    sichtungen.stream()
        .filter(sichtung -> sichtung.gibMelder() == melder)
        .map(sichtung -> sichtung.gibDetails())
        .forEach(details -> System.out.println(details));
}
```

Dies ist äquivalent zu der bisherigen Version. Das andere Beispiel, das wir oben erwähnt hatten – auf die Anzahl der gesichteten Tiere jeder Sichtung abzubilden, sähe so aus:

```
sichtungen.stream()
    .filter(sichtung -> sichtung.gibMelder() == melder)
    .map(sichtung -> sichtung.gibAnzahl())
    . ...
```

Dieses Codesegment würde als Ergebnis einen Stream von ganzen Zahlen liefern, die dann weiter verarbeitet werden könnten.

Übung 5.14 Schreiben Sie die Methode `sichtungenAusgebenVon` in Ihrem Projekt so wie oben beschrieben neu.

Übung 5.15 Schreiben Sie eine Methode, um die Anzahlen aller Sichtungen einer bestimmten Tierart auszugeben. Ihre Methode sollte die `map`-Operation als Teil der Pipeline einsetzen.

Übung 5.16 Wenn eine Pipeline eine `filter`- und eine `map`-Operation enthält, ist die Reihenfolge der Operationen ausschlaggebend für das Endergebnis? Begründen Sie Ihre Antwort.

Übung 5.17 Schreiben Sie die Methode `gefaehrddAusgeben` in Ihrem Projekt neu, sodass sie nun Streams einsetzt, und testen Sie sie. (Um diese Methode zu testen, kann es am einfachsten sein, eine Testmethode zu schreiben, die eine `ArrayList` aus Tiernamen erzeugt und die `gefaehrddAusgeben`-Methode damit aufruft.)

Übung 5.18 *Zusatzaufgabe.* Die Methode `sichtungenAusgebenVon` von `Tiermonitor` enthält die folgende terminale Operation:

```
forEach(details -> System.out.println(details));
```

Ersetzen Sie diese durch

```
forEach(System.out::println);
```

Achten Sie darauf, die korrekte Syntax zu benutzen. Verändert dies die Funktion der Methode? Was können Sie aus Ihrer Antwort ableiten? Wenn Sie sich nicht sicher sind, versuchen Sie herauszufinden, was `::` bei der Verwendung von Lambda-Ausdrücken in Java bedeutet. Wir werden uns dies in den Fortgeschrittenenabschnitten von Kapitel 6 ansehen.

5.5.5 Die Methode `reduce`

Die intermediären Operationen, die wir bisher betrachtet haben, bekommen einen Stream als Eingabe und geben einen Stream als Ausgabe zurück. Manchmal benötigen wir jedoch eine Operation, die ihren Eingabe-Stream auf ein einziges Objekt oder Wert „kollabieren“ lässt, und diese Rolle kommt der `reduce`-Methode als terminaler Operation zu.

Der vollständige Code für das obige Beispiel – alle Tiere eines gegebenen Typs auswählen, diese dann auf die Anzahl der Tiere in jeder Sichtung abbilden und abschließend alle Zahlen addieren – lautet wie folgt:

```
public int gibAnzahl(String tier)
{
    return sichtungen.stream()
        .filter(sichtung -> tier.equals(sichtung.gibTier()))
        .map(sichtung -> sichtung.gibAnzahl())
        .reduce(0, (total, anzahl) -> total + anzahl);
}
```

Wir sehen, dass die Parameter für die **reduce**-Methode ein wenig aufwendiger aussehen als für die **filter**- und **map**-Methoden. Dafür gibt es zwei Gründe:

- Die **reduce**-Methode hat zwei Parameter. Der erste in unserem Beispiel ist 0, der zweite ist ein Lambda-Ausdruck.
- Der hier verwendete Lambda-Ausdruck hat selbst wiederum zwei Parameter, **total** und **anzahl**.

Um die Funktionsweise zu verstehen, ist es ganz hilfreich, wenn wir uns ansehen, wie wir den Code schreiben müssten, wollten wir die Anzahlen auf die traditionelle Art und Weise addieren. Die Schleife sähe dann so aus:

```
public int gibAnzahl(String tier)
{
    int total = 0;
    for(Sichtung sichtung : sichtungen) {
        if(tier.equals(sichtung.gibTier())) {
            total = total + sichtung.gibAnzahl();
        }
    }
    return total;
}
```

Die Berechnung der Summe umfasst die folgenden Schritte:

- Deklariere eine Variable für die Aufnahme des Endwerts und gib ihr den Anfangswert 0.
- Iteriere über der Liste und stelle fest, welche Sichtungsdatensätze sich auf die Tierart beziehen, an der wir interessiert sind (der Filter-Schritt).
- Erhalte die Anzahl der identifizierten Sichtungen (der Map-Schritt).
- Addiere die Anzahl zur Variablen.
- Gib die Summe zurück, die in der Variablen im Verlauf der Iteration summiert wurde.

Der zentrale Punkt, der relevant für das Programmieren einer Stream-basierten Version dieses Prozesses ist, ist zu erkennen, dass die Variable **total** als eine Art „Akkumulator“ fungiert: Jedes Mal, wenn eine relevante Anzahl identifiziert wird, wird diese zu dem Wert addiert, der bisher in **total** angelaufen ist. Dieser neue Wert wird beim nächsten Schleifendurchlauf verwendet. Damit dies funktioniert, muss **total** offensichtlich einen Anfangswert von 0 haben, auf den die erste Anzahl addiert wird.

Die **reduce**-Methode hat zwei Parameter: einen Startwert und einen Lambda-Ausdruck. Formal wird der Startwert *Identität* genannt. Es ist der Wert, mit dem unsere laufende Variable **total** initialisiert wird. In unserem Code haben wir 0 für diesen Parameter übergeben.

Der zweite Parameter ist ein Lambda-Ausdruck mit zwei Parametern: einer für die laufende Variable **total** und einer für das aktuelle Element des Streams. Der Lambda-Ausdruck in unserem Code lautet:

```
(total, anzahl) -> total + anzahl
```

Der Rumpf des Lambda-Ausdrucks sollte den Wert liefern, der durch die Kombination der laufenden Variablen **total** mit dem Element resultiert. In unserem Beispiel heißt „kombinieren“ einfach nur addieren. Die Anwendung dieser **reduce**-Methode bewirkt dann, dass **total** mit null initialisiert wird, jedes Element des Streams hinzugeaddiert wird und zum Schluss **total** zurückgegeben wird.

Da es in unserem Code keine explizite Schleife mehr gibt, ist er anfangs vielleicht ein wenig schwieriger zu verstehen, doch insgesamt werden dieselben Elemente wie in der Version mit der **forEach**-Schleife verwendet.

Übung 5.19 Schreiben Sie Ihre **gibAnzahl**-Methode neu, indem Sie wie hier gezeigt Streams verwenden.

Übung 5.20 Fügen Sie eine Methode zu **Tiermonitor** hinzu, die drei Parameter hat, **tier**, **melderID** und **tagID**, und eine Anzahl zurückgibt, wie viele Sichtungen einer gegebenen Tierart von diesem Melder an einem bestimmten Tag gemacht wurden.

Übung 5.21 Fügen Sie **Tiermonitor** eine Methode hinzu, die zwei Parameter bekommt – **melderID** und **tagID** – und einen **String** zurückliefert, der die Namen der gesichteten Tiere zurückgibt, die vom Melder am gegebenen Tag gesehen wurden. Sie sollten nur Tiere aufnehmen, deren Sichtungszählung größer als null ist, doch Sie müssen sich nicht um doppelt vorkommende Namen kümmern, falls es mehrere nicht leere Sichtungsdatensätze von einem bestimmten Tier gibt. *Hinweis:* Die Verwendung von **reduce** mit **String**-Elementen und einem **String**-Ergebnis ist im Prinzip ganz ähnlich dem Vorgehen mit ganzen Zahlen. Entscheiden Sie sich für die richtige Identität und formulieren Sie einen Lambda-Ausdruck mit zwei Parametern, der die laufende „Summe“ mit dem nächsten Element des Streams kombiniert.

5.5.6 Elemente aus einer Sammlung entfernen

Wir haben gesagt, dass die **filter**-Methode die zugrunde liegende Sammlung, von der der Stream stammt, eigentlich nicht verändert. Prädikat-Lambda-Ausdrücke machen es relativ leicht, alle Elemente aus der Sammlung zu entfernen, die einer bestimmten Bedingung genügen. Nehmen wir zum Beispiel an, wir möchten aus der **sichtungen**-Liste alle **Sichtung**-Datensätze löschen, deren Anzahl null ist. Der folgende Code verwendet die **removeIf**-Methode der Sammlung:

```
/**
 * Entferne aus der sichtungen-Liste alle Datensätze
 * mit Anzahl null.
 * @param melder die ID des Melders
 */
public void entferneNullAnzahl()
{
    sichtungen.removeIf(sichtung -> sichtung.gibAnzahl() == 0);
}
```

Wieder einmal ist die Iteration implizit. Die `removeIf`-Methode übergibt jedes Element der Liste nacheinander dem Prädikat-Lambda-Ausdruck, entfernt dann alle Elemente, für die der Lambda-Ausdruck `true` liefert. Beachten Sie, dass dies eine Methode der Sammlung ist (nicht eines `Streams`) und hiermit die ursprüngliche Sammlung verändert wird.

Übung 5.22 Schreiben Sie die Methode `entferneNullAnzahl` neu, indem Sie die `removeIf`-Methode verwenden, wie oben gezeigt.

Übung 5.23 Schreiben Sie eine Methode `entferneMelder`, der alle Datensätze entfernt, die von einem gegebenen Melder stammen.

5.5.7 Weitere Stream-Methoden

Wir haben gesagt, dass viele der häufig vorkommenden Aufgaben mit diesen drei Operationsarten – *filter*, *map* und *reduce* – bewerkstelligt werden können. In der Praxis bietet Java viele weitere Stream-Methoden, einige davon werden wir später noch kennenlernen. Die meisten sind allerdings lediglich Variationen der drei hier vorgestellten Operationen, auch wenn sie andere Namen haben.

Die Klasse `Stream` hat beispielsweise Methoden namens `count`, `findFirst` und `max` – alles Varianten einer *reduce*-Operation – sowie Methoden namens `limit` und `skip`, die Beispiele einer *filter*-Operation sind.

Übung 5.24 Finden Sie die API-Dokumentation für `Stream` im Paket `java.util.stream` und sehen Sie sich die Methoden `count`, `findFirst` und `max` an. Diese können für jeden Stream aufgerufen werden.

Übung 5.25 Schreiben Sie eine Methode in der Klasse `Tiermonitor`, die einen einzelnen Parameter, `melderID`, bekommt und eine Anzahl zurückgibt, wie viele Sichtsungsdatensätze von diesem Melder gemacht wurden. Verwenden Sie dazu die `Stream`-Methode `count`.

Übung 5.26 Schreiben Sie eine Methode, die den Namen einer Tierart als Parameter hat und die größte Anzahl für dieses Tier in einem einzelnen `Sichtung`-Datensatz zurückliefert.

Übung 5.27 Schreiben Sie eine Methode, die den Namen einer Tierart und eine `melderID` als Parameter hat und das erste `Sichtung`-Objekt liefert, das in der `sichtungen`-Sammlung für diese Kombination gespeichert ist.

Übung 5.28 Lesen Sie sich die Methoden `limit` und `skip` von `Stream` durch und erfinden Sie selbst einige Methoden, um sie zu verwenden.

Zusammenfassung

In diesem Kapitel haben wir einige neue Konzepte eingeführt, die relativ fortgeschritten für dieses Stadium des Buchs sind, aber die sehr eng mit den Konzepten verwandt sind, die wir in Kapitel 4 behandelt haben. Wir haben uns einen *funktionalen Stil* zur Verarbeitung von Datenströmen angesehen. Bei diesem Stil schreiben wir keine Schleifen, um die Elemente in einer Sammlung zu verarbeiten. Stattdessen wenden wir eine *Pipeline* von Operationen auf einen Stream an, der aus einer Sammlung abgeleitet ist, um die Folge in die gewünschte Form zu transformieren. Jede Operation in der Pipeline definiert, wie ein einzelnes Element der Folge behandelt werden soll. Die häufigsten Transformationen werden mittels der Methoden **filter**, **map** und **reduce** durchgeführt.

Operationen in einer Pipeline haben häufig *Lambda-Ausdrücke* als Parameter, um zu spezifizieren, was mit jedem Element der Folge getan werden soll. Ein Lambda-Ausdruck ist eine anonyme Funktion, die keinen oder mehrere Parameter sowie einen Codeblock hat, der dieselbe Rolle wie ein Methodenrumpf spielt.

Wenn Sie sich das Projekt *Tiermonitoring-v1* ansehen, mit dem Sie gearbeitet haben, dann haben Sie vielleicht bemerkt, dass es noch zwei Methoden gibt, die **forEach**-Schleifen verwenden und die wir bisher noch nicht für die Verwendung von Streams umgeschrieben haben. Dies sind Methoden, die neue Sammlungen als Methodenergebnis zurückgeben; dieses Thema – eine neue Sammlung aus einem Stream zu erzeugen – haben wir noch nicht behandelt. Wir kommen darauf in Kapitel 6 zurück, wo wir sehen werden, wie eine neue Sammlung aus der transformierten Folge gewonnen werden kann.

Zentrale Konzepte in diesem Kapitel: Funktionale Programmierung, Lambda-Ausdruck, Stream, Pipeline, **filter**, **map**, **reduce**





Zusammenfassung der Konzepte

- **Lambda-Ausdruck** Ein Lambda-Ausdruck ist ein Codesegment, das für eine spätere Ausführung gespeichert werden kann.
- **Funktionaler Stil** Im funktionalen Stil der Verarbeitung von Sammlungen entnehmen wir nicht jedes Element einzeln, um damit zu arbeiten. Stattdessen übergeben wir der Sammlung ein Codesegment, das auf jedes Element angewandt wird.
- **Streams** Streams vereinheitlichen die Verarbeitung von Elementen einer Sammlung und anderer Datenmengen. Ein Stream bietet nützliche Methoden, um diese Datenmengen zu manipulieren.
- **filter** Wir können eine **filter**-Operation auf einen Stream anwenden, um nur bestimmte Elemente auszuwählen.
- **map** Wir können einen Stream mithilfe der **map**-Operation auf einen anderen Stream abbilden, wobei jedes Element des ursprünglichen Streams durch ein neues Element ersetzt wird, das aus dem Original abgeleitet ist.
- **reduce** Wir können einen Stream mithilfe der **reduce**-Operation reduzieren, das heißt, wir wenden eine Funktion an, die einen ganzen Stream entgegennimmt und einen einzelnen Ergebniswert zurückgibt.
- **Pipeline** Eine Pipeline ist die Kombination von zwei oder mehreren Stream-Funktionen in einer Kette, wobei jede Funktion der Reihe nach angewendet wird.

Übung 5.29 Erstellen Sie eine Kopie des Projekts *Musiksammlung-v5* aus Kapitel 4. Schreiben Sie die Methoden `alleTracksAusgeben` und `bestimmteTracksAusgeben` der Klasse `MusikSammlung` neu, um Streams und Lambda-Ausdrücke zu verwenden.



KAPITEL

6

Bibliotheksklassen nutzen

Lernziele

Zentrale Konzepte in diesem Kapitel: Mit Bibliotheksklassen umgehen, Dokumentation lesen, Dokumentation schreiben

Java-Konstrukte in diesem Kapitel: `String`, `Random`, `HashMap`, `HashSet`, `Iterator`, `static`, `final`, Autoboxing, Wrapper-Klassen

In Kapitel 4 haben wir die Klasse `ArrayList` aus der Java-Klassenbibliothek vorgestellt. Wir haben gesehen, dass wir mit dieser Klasse Dinge tun können, die sonst nur recht schwierig umzusetzen wären (in unserem Beispiel eine beliebige Anzahl von Objekten speichern).

Dies war nur ein Beispiel für eine nützliche Klasse aus der Java-Bibliothek. Die Bibliothek besteht aus Tausenden von Klassen, von denen viele sehr nützlich für Ihre Arbeit sein können, während Sie vermutlich viele andere niemals benötigen werden.

Für einen guten Java-Programmierer ist es wichtig, kompetent mit der Java-Bibliothek arbeiten zu können. Vor allem die Fähigkeit, die richtigen Klassen aus ihr auszuwählen, ist unerlässlich. Sobald Sie mit der Bibliothek einigermaßen vertraut sind, werden Sie merken, dass Sie vieles sehr viel einfacher und schneller mithilfe der Bibliotheksklassen erledigen können als ohne sie. Der Umgang mit diesen Klassen ist der Schwerpunkt in diesem Kapitel.

Bei den Elementen in der Bibliothek handelt es sich nicht um einen Satz von beliebigen Klassen, die in keinem Zusammenhang stehen und die wir alle einzeln lernen müssen, sondern sie werden oft nach bestimmte Kriterien zusammengestellt, die auf Gemeinsamkeiten beruhen. Auch hier haben wir es wieder mit dem Konzept der Abstraktion zu tun, die uns hilft, eine Fülle an Informationen zu verarbeiten. Zu den wichtigsten Teilen der Bibliothek gehören die Sammlungen, zu denen auch die `ArrayList` gehört. Wir werden in diesem Kapitel weitere Sammlungen besprechen und feststellen, dass sie viele Attribute gemeinsam haben, sodass wir oft von den individuellen Details einer speziellen Sammlung abstrahieren und über Sammlungsklassen im Allgemeinen sprechen können.

Wir werden neue Sammlungsklassen sowie einige andere nützliche Bibliotheksklassen vorstellen und diskutieren. Im Verlauf dieses Kapitels werden wir uns mit der Erstellung einer einzelnen Anwendung beschäftigen (einem System für einen